

EDM
HOLDINGS

EDM OS Technical Blueprint

Architecture, data model, security and the phased build — for the engineering team delivering the platform.

EDM OS — construction operating system · UAE fit-out & drywall

Stack: Next.js · NestJS · PostgreSQL/Prisma · Supabase · Vercel · n8n · Anthropic/OpenAI

Companion: the **edm-os** foundation repository (schema, auth, RBAC, reference modules)

Designed to scale from AED 0 to AED 500m turnover

Contents.

01	Build vs buy — read first	The honest commercial call
02	System architecture	The stack and how it fits
03	Module map	Core, trade and AI modules
04	Data model	The entities and relationships
05	API design	Conventions and the reference modules
06	Roles & permissions	Who can do what
07	AI integration	Five assistants, done safely
08	Security framework	Tenancy, auth, data, compliance
09	Mobile	Field app and offline
10	Hosting & deployment	Where it runs, how it ships
11	Phased roadmap	MVP → Commercial → Enterprise

The honest commercial call.

Before a line of this is built, the decision has to be made with open eyes. A platform that genuinely rivals Procore is a multi-year programme costing seven figures in engineering. EDM today is pre-revenue with no live UAE projects. So the recommendation depends entirely on intent.

If EDM OS is an internal tool

At AED 0 revenue running zero projects, build nothing custom yet. Run on off-the-shelf (Buildertrend or a lean Monday/Notion + accounting + an estimating tool), and revisit a custom build at ~AED 30–50m turnover when process pain justifies the cost.

If EDM OS is a product

A vertical SaaS for UAE fit-out contractors is a legitimate venture with a real gap in the market. Then this blueprint and the foundation repo are exactly right — build the MVP, win design partners, and sell it. The data model is already multi-tenant for this reason.

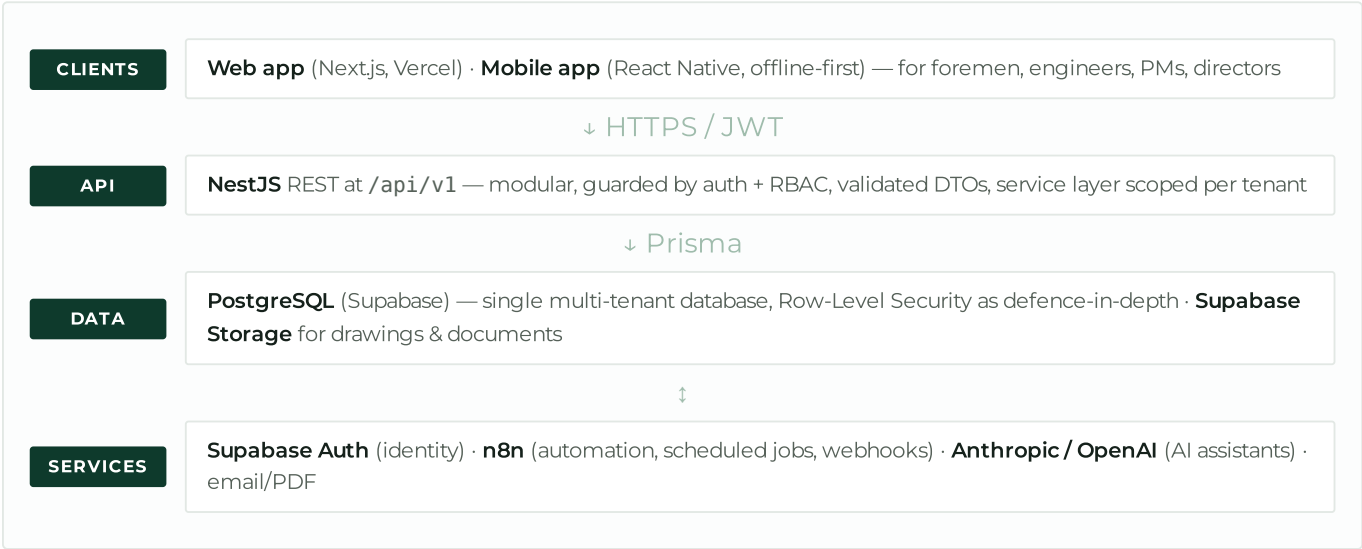
Either way, this is the correct first artifact. The blueprint, the complete data model and the reference-module foundation de-risk the build and let a senior team start immediately — without committing to the full programme up front. Build Phase 1, learn, then decide on Phases 2–3.

What the companion repository contains

- **The complete data model** — every module as production Prisma schema.
- **Authentication and RBAC** — Supabase JWT verification + role guards, applied globally.
- **Two reference modules** — Tenders and Projects, built end-to-end (controller, service, DTOs, tenant scoping) as the exact pattern for the rest.
- **The web foundation** — the design system and a working Executive dashboard, Tenders and Projects screens.

The stack, and how it fits.

A modular monolith to start — one API, one database, clean module boundaries — which is the right call for a small team and refactors cleanly to services later if scale demands. The chosen stack is boring on purpose: proven, hireable, and fast to build on.



Why these choices

- **Next.js + NestJS + Postgres** — a mainstream TypeScript stack end to end; one language, a deep hiring pool, and excellent tooling.
- **Supabase** — auth, Postgres and storage in one managed platform; removes a pile of undifferentiated work at the start.
- **Prisma** — type-safe data access and migrations; the schema is the single source of truth.
- **Modular monolith first** — services add operational cost a small team can't absorb. Split out only the modules that genuinely need it, later.

What the platform is made of.

Core modules

MODULE	DOES	PHASE
Executive dashboard	Live pipeline, cash, profit, utilisation, approvals — the owner's single view.	1
CRM	Companies, contacts, leads, opportunities, activities, follow-ups.	1
Tenders	Tender register, status workflow, history, weighted pipeline.	1
Estimating	Cost database, rates, BOQ lines, markups, quotations.	1
Projects	Programme, phases, milestones, tasks, dependencies, dashboards.	1
Site reporting	Daily reports — labour, plant, deliveries, weather, delays.	1
Variations	VO workflow, financial impact, logs.	2
RFIs	Raise, respond, track, attach, overdue flags.	2
Quality	Inspections, checklists, snags, NCRs, corrective actions.	2
HSE	Incidents, toolbox talks, inspections, risk assessments.	2
Procurement	Suppliers, POs, material requests, deliveries, spend.	2
Document control	Drawings/contracts/specs, revisions, approvals.	2
Financials	Budgets, cost codes, commitments, invoices, retentions, forecasting.	3

Specialised trade modules

One flexible TradeProgress model captures installed quantities per trade with typed columns plus a JSON attributes field for trade-specifics — so drywall, ceilings, joinery, cladding, glazing and painting each track what matters without six rigid tables.

Drywall — m ² installed, board type, fire/acoustic rating, partition type	Ceilings — grid & tile system, quantities
Joinery — production / delivery / installation stage, approvals	Timber cladding — material, fixing system, area
Aluminium & glazing — fabrication / delivery / install, system	Painting — prep / primer / finish, coats, quantities

AI modules

Estimating take-off · RFI drafting · progress reporting · proposal generation · management advisor — all logged through one AiJob model (see §07).

The backbone.

Roughly fifty tables, every business table scoped by `organisationId` for multi-tenancy. The full schema is in the repo at `packages/db/prisma/schema.prisma`; the core relationships are below.

Core entities & how they connect

- **Organisation** is the tenant root — every record hangs off it. **User** belongs to an organisation and holds a **Membership** with a **Role**.
- **CRM**: Company → Contact; Lead → Opportunity → (becomes) **Tender**.
- **Tender** links to Company three ways (client, consultant, main contractor), carries **Estimates** (with `EstimateLines` and `Quotations`), and converts to a **Project**.
- **Project** is the delivery hub: Phases, Milestones, Tasks (with dependencies), `DailyReports` (Labour/Plant/Delivery entries), Variations, RFIs, Inspections (→ Snags), NCRs, Incidents, POs, Budgets, Invoices, Documents and `TradeProgress`.
- **Document** has versioned **DocumentRevisions** pointing at Storage keys, and attaches to Project, Tender, Variation or RFI.
- **AuditLog** and **Notification** span everything for traceability and alerts.

Conventions

Keys	<code>cuid()</code> primary keys; composite unique keys per tenant (e.g. [<code>organisationId</code> , <code>tenderNo</code>]).
Money	<code>Decimal(14,2)</code> for values, <code>Decimal(12,4)</code> for rates; currency defaults to AED.
Enums	Status fields are Postgres enums (<code>TenderStatus</code> , <code>ProjectStatus</code> , <code>VariationStatus</code> ...) — not free text.
History	Status changes write immutable event rows (e.g. <code>TenderEvent</code>); records are append-then-update, never silently overwritten.
Indexes	On <code>organisationId</code> , status and date fields that drive list and dashboard queries.

Conventions, shown in code.

A versioned REST API under `/api/v1`. Every route is authenticated and role-guarded by default; tenancy is taken from the verified token, never from the request body. The Tenders and Projects modules in the repo are the canonical pattern.

The rules every module follows

- **Auth by default.** A global JWT guard protects all routes; a route opts out only with `@Public()` (e.g. health).
- **RBAC on writes.** Mutations carry `@Roles(...)`; the guard rejects insufficient roles.
- **Tenant scope in the service.** Every query filters by `organisationId` from `AuthContext` — the boundary is enforced server-side, in one place.
- **Validated DTOs.** `class-validator` on every input; a global pipe strips unknown fields.

Reference: the tender status transition

```
@Patch("/:id/status")
@Roles(Role.OWNER, Role.DIRECTOR, Role.COMMERCIAL_MANAGER)
moveStatus(@CurrentUser() u, @Param("id") id, @Body() dto) {
  return this.tenders.moveStatus(u.organisationId, id, dto.status, dto.note);
}

// service – ownership check, then atomic update + history event
async moveStatus(orgId, id, status, note) {
  await this.get(orgId, id); // 404 if not this tenant's
  return this.prisma.$transaction([
    this.prisma.tender.update({ where: { id }, data: { status } }),
    this.prisma.tenderEvent.create({ data: { tenderId: id, status, note } }),
  ]);
}
```

Selected endpoints already implemented: GET/POST/PATCH/DELETE `/tenders`, GET `/tenders/pipeline` (weighted), PATCH `/tenders/:id/status`; GET/POST/PATCH `/projects`, GET `/projects/:id/financials`.

Who can do what.

Twelve roles, including external parties (client, consultant, main contractor) who get scoped, read-mostly access to the projects that concern them. Permissions are enforced per endpoint; the matrix below is the policy.

ROLE	TYPICAL ACCESS
Owner / Director	Everything, including financials, deletes and settings.
General Manager	All operational modules; create projects; no destructive settings.
Commercial Manager	Tenders, estimating, variations, financials (commercial), CRM.
Estimator	Estimating and tenders; read projects.
Project Manager	Owned projects end-to-end — programme, RFIs, variations, quality, site.
Site Engineer	Site reports, quality, RFIs, HSE on assigned projects.
Foreman	Daily reports, labour/plant, trade progress, snags — mostly via mobile.
Administrator	User management, documents, master data; no commercial figures.
Client	Read access to their project's progress, RFIs and approvals only.
Consultant / Main contractor	Scoped access to shared RFIs, submittals and documents on relevant projects.

Defence in depth: roles are enforced in the API, and Postgres Row-Level Security is recommended as a second layer so a query can never cross tenants even if application code has a bug.

Five assistants, done safely.

AI is additive, not autonomous. Each assistant produces a draft a human reviews; nothing is sent to a client or committed to the record without sign-off. Every call runs as an **AiJob** — queued, logged, costed, auditable.

ASSISTANT	INPUT → OUTPUT	MODEL FIT
Estimating take-off	Drawing / BOQ → identified quantities, materials, labour for review.	Vision + reasoning
RFI drafter	Issue note → a clear, properly-worded RFI draft.	Anthropic / OpenAI
Progress reporter	Site data → daily/weekly/monthly report narrative.	Fast model
Proposal generator	Project facts → quotation, capability statement, method statement drafts.	Reasoning
Management advisor	Cash, delays, productivity, profit → flagged risks and recommendations.	Reasoning

Guardrails

- **Human-in-the-loop** on everything client-facing or financial. AI drafts; a person signs off.
- **Grounded** in the org's own data; no figures or claims invented — consistent with how EDM operates everywhere else.
- **Logged & costed** per job (tokens in/out, provider, model) for spend control and audit.
- **Tenant-isolated** — a job only ever reads its own organisation's data.

Trust, built in.

Tenancy isolation Every business table carries organisationId, filtered in the service layer, with Postgres RLS as a second layer. No query crosses tenants.	Authentication Supabase-issued JWTs verified on every request; short-lived access tokens; MFA available for privileged roles.
Authorisation Role-based guards on all mutations; least-privilege defaults; external parties scoped to their projects.	Data protection TLS in transit; encryption at rest (Supabase); signed, expiring URLs for document downloads from Storage.
Auditability AuditLog records create/update/delete/export with before/after and actor; immutable status-event history.	Operational Secrets in environment/secret manager (never in code); rate limiting; automated daily backups with point-in-time recovery.

UAE data note. Confirm data-residency requirements for the clients and projects EDM serves; Supabase offers regional hosting. For government or sensitive work, residency and access controls should be agreed before storing project data.

Built for the field.

Site teams don't sit at desks and UAE sites don't always have signal, so the mobile app is offline-first and deliberately narrow — the few things a foreman or engineer actually does on site, done fast.

Who & what

Foremen and site engineers: daily reports, labour/plant, trade progress (m² installed), snags with photos, RFIs, toolbox talks.
PMs and directors: dashboards and approvals on the move.

How it's built

React Native (shared TypeScript with the web). Local store (SQLite/WatermelonDB) queues entries offline and syncs when signal returns; camera and photo capture native.

- **Offline-first** — capture works with no signal; a sync engine reconciles on reconnect (last-write-wins with conflict flags).
- **Photo-led** — snags and progress are mostly a photo plus a tap, feeding the same records the web uses.
- **One codebase of types** — DTOs shared with the API so mobile and web never drift.

Where it runs, how it ships.

COMPONENT	HOSTING	NOTES
Web (Next.js)	Vercel	Preview deploys per branch; edge CDN; instant rollbacks.
API (NestJS)	Render / Fly.io / Railway (or a container on AWS)	Containerised; horizontal scaling when load needs it.
Database + Auth + Storage	Supabase	Managed Postgres, auth and object storage; daily backups + PITR.
Automation	n8n (managed or self-hosted)	Scheduled jobs, webhooks, notifications.

Pipeline

- **Environments:** local → staging → production, each with its own Supabase project and secrets.
- **CI/CD:** GitHub Actions — lint, type-check, test, then deploy; `prisma migrate deploy` runs on release.
- **Migrations:** versioned in the repo; never edited by hand against production.
- **Observability:** error tracking (Sentry), uptime checks, structured logs from day one.

Cost shape at the start: the managed stack (Vercel + Supabase + a small API host + n8n) runs at low hundreds of dirhams a month until there's real usage, and scales with it — appropriate for a platform starting at zero.

MVP → Commercial → Enterprise.

PHASE	SCOPE	OUTCOME
Phase 1 — MVP BUILD FIRST	Auth + RBAC, CRM, Tenders, Estimating, Projects, Site reporting, Executive dashboard. The foundation repo covers auth, the data model and Tenders + Projects already.	EDM runs its pipeline, estimates and live projects in one place.
Phase 2 — Commercial NEXT	Variations, RFIs, Quality, HSE, Procurement, Document control, the trade modules, and the offline mobile app.	Full project delivery and site operations covered; field teams on mobile.
Phase 3 — Enterprise LATER	Financials & forecasting, the AI suite, advanced analytics, SSO, and — if productised — multi-organisation SaaS, billing and integrations.	A scalable platform; optionally a sellable product for other UAE contractors.

Each phase ships with

- Defined features and acceptance criteria · the relevant slice of the data model (already designed) · API modules following the reference pattern · UI on the existing design system · a test plan (unit + e2e on the critical paths) · a deployment to staging then production.
- The discipline that makes it work:** ship Phase 1, put it in EDM's hands on real projects, and let what you learn shape Phases 2–3. A platform built against real use beats one built against a wishlist — and it's how the good ones, Procore included, actually grew.

EDM OS · Construction operating system for EDM Holdings · Dubai, UAE
Companion: the edm-os foundation repository (schema, auth, RBAC, Tenders + Projects, web design system + Executive dashboard).
This is the Phase-0 blueprint and foundation — the start of a build, scoped honestly, not a finished product.